

Introduction to Retrieval-Augmented Generation

Retrieval-Augmented Generation, or RAG, is a technique that combines a retrieval system with a language model. Instead of relying only on what the language model memorized during training, the system first searches an external knowledge base for relevant information, then passes that information to the model as context before it generates an answer.

Why RAG Matters

Large language models have two main limitations. First, their knowledge is frozen at the time training data was collected, so they cannot know about new documents or recent events. Second, they can hallucinate, meaning they sometimes generate confident but incorrect information. RAG addresses both problems by grounding the model's answer in real documents that are retrieved at query time.

How Chunking Works

Before documents can be searched efficiently, they must be split into smaller pieces called chunks. A common strategy is to split text into chunks of a few hundred words, with some overlap between consecutive chunks so that context is not lost at the boundaries. Chunk size is a tradeoff: chunks that are too small lose context, while chunks that are too large make retrieval less precise.

Embeddings and Vector Databases

An embedding model converts each chunk of text into a numerical vector that captures its meaning. Chunks with similar meaning end up with vectors that are close together in this vector space. These vectors are stored in a vector database such as FAISS or ChromaDB, which is optimized for finding the nearest vectors to a given query very quickly.

The Retrieval Step

When a user asks a question, the question itself is converted into an embedding using the same embedding model. The vector database is then searched for the chunks whose embeddings are closest to the question's embedding. These are the chunks considered most relevant to answering the question.

The Generation Step

The retrieved chunks are inserted into a prompt along with the original question, and this combined prompt is sent to a language model. The model is instructed to answer using only the information found in the provided chunks. This final step is called generation, and it is what completes the Retrieval-Augmented Generation pipeline.

Controlling Hallucination

Even with retrieval, a language model can still hallucinate if it ignores the provided context or fills gaps with invented facts. Common mitigation strategies include explicitly instructing the model to say it does not know when the answer is not in the context, showing the model the source of each chunk so answers can be verified, and keeping retrieved chunks short and directly relevant rather than overly broad.